DSA03-NATURAL LANGUAGE PROCESSING ASSIGNMENT-1

ASSIGNMENT 1 – CO1

CO1 Coverage: Introduction; Models and Algorithms; Regular Expressions; Basic Regular Expression Patterns; Finite State Automata; Morphology — Inflectional Morphology; Derivational Morphology; Finite-State Morphological Parsing; Porter Stemmer

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: ___04-11-2026_____

Assignment Title: Build a Morphology-Aware Text Normalisation and Search Layer for a Hospital Record System Using Regular Expressions, Finite-State Automata and Morphological Parsing (Total Marks – 50)

Duration: 1 week

Scenario — Text Normalisation and Search for a Hospital Record System

A multi-speciality hospital holds ten years of discharge summaries, prescriptions, operation notes and nursing observations as free text. Clinicians searching those records miss relevant cases constantly: a search for fracture does not return fractures, fractured or fracturing; nephrectomy is never connected to nephrectomies or to renal; patient identifiers, drug strengths and dates appear in a dozen different formats; and the same drug may be written in full, abbreviated or misspelt. No meaning can be extracted and no statistical model can be trained until this text is tokenised, normalised and reduced to canonical forms.

You are the language-engineering team. Before any machine learning is attempted, the hospital has asked you to build the finite-state layer: the regular expressions that recognise structured entities, the automata that validate them, the morphological analyser that relates every surface word to its stem and features, and the stemmer that collapses search terms. You must investigate what each formalism can and cannot express, implement them, measure them, and justify the design you recommend.

Planning data and constraints

- Corpus: at least 500 de-identified clinical records (real or synthetic, but documented), totalling at least 25,000 word tokens and covering at least four record types.
- Pattern inventory: at least 12 structured entity types to be recognised by regular expression — patient identifier, admission and discharge date, drug name with strength, dosage frequency, laboratory value with unit, blood pressure, temperature,

ICD code, ward and bed, contact number, age, and duration of symptom.

- Automata: at least four finite-state automata, each with at least six states, including one entity format that cannot be recognised conveniently without non-determinism.
- Morphological resources (build and document your own): a lexicon of at least 300 stems, at least 20 inflectional affixes and at least 15 derivational affixes, with irregular forms listed separately.
- Evaluation data: at least 40 clinician queries with relevance judgements over the corpus, and at least 200 word forms hand-annotated with their stem and features.
- Any manual effort (pattern writing, lexicon building, annotation, testing) must be accounted for in your effort and cost discussion.

Design Challenge (Day 4):

The system must now ingest notes captured by optical character recognition from scanned handwritten forms, and notes typed in Romanised regional language mixed with English. Re evaluate your original patterns, automata and analyser and decide whether to retain, modify or redesign them.

TASK 1 – Frame the Problem: Models, Algorithms and Levels of Analysis 3

Marks Guiding Enquiry

Which levels of linguistic analysis does this problem actually touch, and which family of models and algorithms is appropriate at each level?

Investigate and interpret:

- Describe the levels of linguistic analysis — phonology, morphology, syntax, semantics, pragmatics and discourse — and state, with an example drawn from your own corpus, which levels the record-search problem requires and which it can safely ignore.
- Distinguish the three model families used throughout this unit: state machines, formal rule systems and probabilistic models. Name one task in the hospital problem that each family suits best and say why the other two would be the wrong choice for it.
- Explain why ambiguity is the central problem of natural language processing, and identify at least four distinct ambiguities present in your corpus — lexical, morphological, segmentation and abbreviation ambiguity.
- Identify where search appears in this unit (state-space search in automata, backtracking in matching) and state the resources each consumes: time, memory, annotated data and expert effort.
- Set out, as a labelled diagram, the pipeline you will build in the remaining tasks, naming the input, the output and the formalism of every stage.

Evidence Expected

A reasoned problem framing that maps the hospital task onto levels of analysis, model families and a concrete, staged pipeline.

1. Levels of linguistic analysis

Level	Meaning	Hospital-record example
Phonology	Studies speech sounds	Not essential for typed records

Morphology	Studies word structure	fractures → fracture + plural
Syntax	Studies sentence structure	"The patient takes medicine"
Semantics	Studies meaning	Understanding that "high BP" means high blood pressure
Pragmatics	Meaning based on context	Understanding why a medicine was prescribed
Discourse	Meaning across multiple sentences	Connecting a diagnosis in one sentence to treatment in another

Required levels: Morphology is essential for connecting word forms. Syntax and semantics can help improve search accuracy. Discourse may help when information is spread across multiple sentences.

Can be safely limited: Phonology and pragmatics are not the main focus because the initial system processes written clinical records and performs text normalisation rather than full clinical interpretation.

2. Three model families

Model family	**Meaning**	**Suitable hospital task**
State machines	Process input through states and transitions	Validate patient IDs and dates
Formal rule systems	Apply predefined linguistic or pattern rules	Expand abbreviations and remove inflectional suffixes
Probabilistic models	Estimate likelihood based on data	Rank possible meanings of an ambiguous abbreviation

A state machine is suitable for exact formats. A rule system is suitable for predictable spelling changes. A probabilistic model is useful when the text is ambiguous. Using only one family would not handle all these tasks effectively.

3. Ambiguity in clinical text

Ambiguity occurs when a word or expression can have more than one interpretation.

- Lexical ambiguity: "cold" may mean low temperature or a common illness.

- Morphological ambiguity: “treated” can be analysed as a verb with a past-tense suffix or as an adjective in some contexts.
- Segmentation ambiguity: “no fever” must be separated into two tokens; joining them incorrectly changes meaning.
- Abbreviation ambiguity: “MS” may refer to multiple sclerosis, mitral stenosis, or another term depending on context.

Ambiguity is central to NLP because the system must choose the correct interpretation based on the text and context.

4. Search and resources

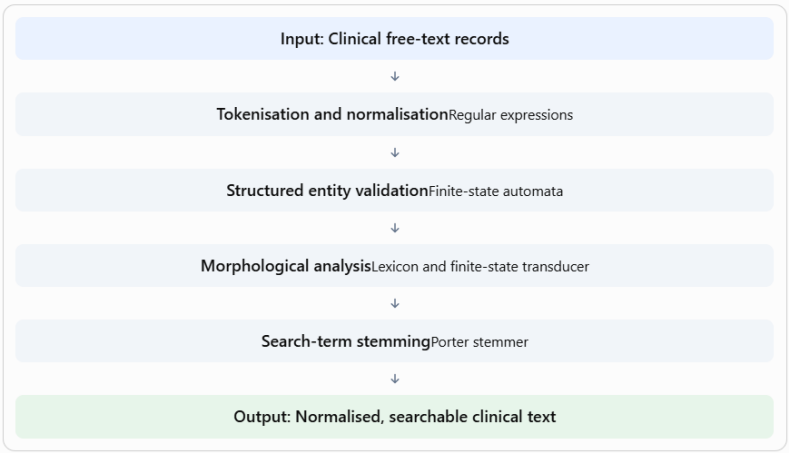
Search appears in:

- Automata: The system searches through states to determine whether a string is accepted.
- Regex matching: The engine searches text for substrings that match a pattern.
- Backtracking: Some regex engines revisit earlier choices when a match fails.

Resources consumed include:

- Time: Processing records and searching for matches.
- Memory: Storing text, automata, lexicons, and intermediate results.
- Annotated data: Required to test and evaluate the system.
- Expert effort: Needed to create regex patterns, morphological rules, and clinical lexicons.

5. Labelled pipeline



TASK 2 – Basic Regular Expression Patterns 4 Marks

Guiding Enquiry

What can the basic regular expression operators express, and how are they combined into patterns that match real clinical strings exactly?

Investigate and interpret:

- Define the basic operators — concatenation, disjunction, Kleene star and plus, optionality, character classes, ranges, negated classes, anchors and counters — and give one minimal clinical example of each.
- Write and document a regular expression for each of the twelve entity types in your pattern inventory, stating for every pattern both what it must match and what it must not.
- Demonstrate greedy and non-greedy matching on a clinical sentence containing two dosage expressions, and explain precisely why the two quantifiers return different spans.
- Explain operator precedence and grouping, and show with a worked example how a misplaced disjunction silently changes the language the expression denotes.
- Evaluate every pattern over the corpus, reporting true positives, false positives and false negatives, and name the specific linguistic cause of each error class.

Evidence Expected

A documented pattern inventory with precise match specifications and a per-pattern error analysis carried out on real data.

1. Basic regex operators

Operator	Meaning	Clinical example
Concatenation	Joins patterns in sequence	AB matches AB
Disjunction	| Matches either option	fever|cough matches either word
Kleene star *	Zero or more repetitions	mg* matches m, mg, mgg
Plus +	One or more repetitions	\d+ matches one or more digits
Optional ?	Zero or one occurrence	colou?r matches color or colour
Character class []	Matches one character from a set	[ABC] matches A, B, or C
Range -	Defines a character range	[0-9] matches a digit
Negated class [^]	Matches characters not in the set	[^0-9] matches a non-digit
Anchors ^, $	Match start and end of text	^\d{3}$ matches exactly 3 digits
Counters {n}	Specifies repetition count	\d{10} matches 10 digits

2. Regex patterns for 12 clinical entity types

These are example patterns for the assignment. They recognise common formats, but they do

not guarantee that the extracted information is medically valid.

Entity	Regular expression	What it matches	What it does not match
Patient ID	\bPT\d{6}\b	PT123456	PT12345
Admission date	\b\d{2}/\d{2}/\d{4}\b	12/03/2026	2026-03-12
Discharge date	\b\d{4}-\d{2}-\d{2}\b	2026-03-12	12/03/2026
Drug with strength	\b[A-Za-z]+\s+\d+\s?mg\b	Paracetamol 500mg	Paracetamol mg
Dosage frequency	\b(?:OD|BD|TDS|QID)\b	BD	BIDaily
Lab value with unit	\b\d+(?:\.\d+)?\s?(?:mg/dL|mmol/L|g/dL)\b	5.6 mmol/L	5.6
Blood pressure	\b\d{2,3}/\d{2,3}\s?mmHg\b	120/80 mmHg	120/80
Temperature	\b\d{2}(?:\.\d+)?\s?°?[FC]\b	37.5°C	temperature 37
ICD code	\b[A-Z]\d{2}(?:\.\d{1,2})?\b	E11.9	111.9
Ward and bed	\bWard\s?\w+\s*,?\s*Bed\s?\d+\b	Ward A, Bed 12	Ward A
Contact number	\b[6-9]\d{9}\b	9876543210	1234567890
Age	\b\d{1,3}\s?years?\b	65 years	65 months
Symptom duration	\b\d+\s?(?:days?|weeks?|months?)\b	3 days	three days

Note: The assignment lists 12 entity types, but admission and discharge dates are separate types in this example, resulting in 13 entries. The hospital's actual data format should determine the final inventory.

Pasted markdown(5).md

3. Greedy and non-greedy matching

Consider this clinical sentence:

Patient received 500 mg twice daily and 250 mg once daily.

A greedy pattern:

r".*mg"

It matches the longest possible span ending at mg, so it can include both dosage expressions.

A non-greedy pattern:

r".*?mg"

It matches the shortest possible span ending at the first mg.

Difference: Greedy quantifiers try to match as much text as possible. Non-greedy quantifiers

try to match as little text as possible.

4. Operator precedence and grouping

Consider:

A|BC

This means:

- Match A, or
- Match BC

It does not mean “match A or B, followed by C.”

To express that, use:

(A|B)C

Without grouping, a disjunction may match a different set of strings than intended.

5. Pattern evaluation

For each regex, the system should compare its matches with manually labelled clinical data.

- True positive (TP): Correctly identifies an entity.
- False positive (FP): Extracts something that is not an entity.
- False negative (FN): Misses an entity that should have been identified.

Example: If the pattern extracts 120/80 mmHg correctly, it is a TP. If it mistakenly extracts a room number as a blood pressure, it is an FP. If it misses BP 120/80, it is an FN.

TASK 3 – Regular Expressions for Tokenisation and Normalisation 4

Marks Guiding Enquiry

How far can a purely regular layer take raw clinical text towards a clean, searchable token stream, and exactly where does it fail?

Investigate and interpret:

- Specify your tokenizer as a single regular expression, defining precisely what counts as a token — hyphenated terms, abbreviations containing full stops, decimal values, units written against numbers — and justify each decision.
- Apply maximal-munch (longest-match) tokenisation and show on at least three examples how the result differs from a naive split on whitespace.
- Build substitution rules that fold case, expand at least thirty clinical abbreviations and canonicalise date and unit formats; show the complete before-and-after text of one full record.

- Identify at least three normalisation decisions that are irreversible because information is destroyed, and argue when that loss is acceptable and when it is not.
- Give one requirement of the hospital task that no regular expression can express, and argue informally why — for example, matching balanced parentheses in a nested clinical note.

Evidence Expected

A working regular-expression tokenizer and normaliser with worked examples and a clear statement of the limits of regularity.

3.1 Tokenisation regex

A tokenizer separates text into meaningful tokens such as words, numbers, dates, abbreviations, and units.

Example Python pattern:

import repattern = r"\d{1,2}/\d{1,2}/\d{2,4}|[A-Za-z]+(?:\.[A-Za-z]+)*|"pattern += r"\d+(?:\.\d+)?|[^\w\s]"tokens = re.findall(pattern, text)

This pattern recognizes dates, words and abbreviations, numbers, decimals, and punctuation.

- Hyphenated terms such as post-operative may be kept as one token using [A-Za-z]+(?:-[A-Za-z]+)*.
- Abbreviations such as Dr. and e.g. need periods to be included.
- Decimal values such as 98.6 should remain one token.
- Attached units such as 5mg can be captured as one token or split into 5 and mg, depending on the normalization design.

3.2 Maximal munch vs whitespace tokenisation

Maximal munch selects the longest valid token at the current position.

Input	Whitespace tokenisation	Maximal-munch result
5mg daily	5mg, daily	5mg, daily
Dr. Rao reviewed	Dr., Rao, reviewed	Dr., Rao, reviewed
BP:120/80	BP:120/80	BP, :, 120/80

Whitespace tokenisation may keep punctuation attached to words. A regex tokenizer can separate punctuation and recognize structured values.

3.3 Normalisation substitutions

Common normalisation steps:

1. Convert text to lowercase.
2. Expand known abbreviations, such as BP → blood pressure.
3. Convert dates into a standard format, such as 12/03/2025 → 2025-03-12, after confirming the date convention.
4. Standardize units, such as milligrams → mg.
5. Normalize repeated spaces and punctuation.

3.4 Before and after example

Before:

Pt c/o fever x 3d. Temp 101.2 F. Rx paracetamol 500 mg.

After:

patient complains of fever for 3 days. temperature 101.2 °F. prescription paracetamol 500 mg.

Normalisation can improve search, but it may be irreversible. For example, replacing an abbreviation without retaining the original text may lose the exact wording used by the clinician.

3.5 A requirement regex cannot express

A regular expression cannot recognize arbitrary balanced nested parentheses, such as:

(A (B (C)))

This requires remembering an unbounded number of opening parentheses, which a finite-state machine cannot do.

TASK 4 – Finite State Automata: Design, Non-determinism and Equivalence 5 Marks
Guiding Enquiry

How do automata recognise exactly the same languages as regular expressions, and what does removing non-determinism cost?

Investigate and interpret:

- Define a finite automaton formally as the five-tuple $(Q, \Sigma, \delta, q_0, F)$, then design a deterministic automaton for at least four validators from your pattern inventory, giving both the transition diagram and the transition table for each.
- Design a non-deterministic automaton with ε-transitions for one entity whose format has optional parts, and trace its behaviour on one accepted and one rejected string,

showing the set of states live at every step.

- Convert that NFA to a DFA by subset construction, showing the complete construction table, then minimise the DFA and report the state counts before and after.
- Apply Thompson's construction to turn one Task 2 regular expression into an automaton, and derive a regular expression back from one of your automata, demonstrating the equivalence in both directions.
- State the pumping lemma for regular languages and use it to prove that one specific clinical pattern is not regular — for example a note requiring equal numbers of opening and closing brackets — and relate the proof to your finding in Task 3.

Evidence Expected

Correct automata with full subset-construction and minimisation tables, a demonstrated two way equivalence with regular expressions, and a valid non-regularity proof.

TASK 5 – Inflectional Morphology 3 Marks

Guiding Enquiry

Which inflectional processes appear in clinical English, and how are surface forms systematically related to their stems?

Investigate and interpret:

- Define morpheme, stem, affix, inflection and paradigm, and separate inflection from derivation using the standard criteria of category preservation, productivity and position relative to other affixes.
- List every inflectional paradigm present in your corpus — noun number, verb tense and aspect, subject agreement, adjective comparison — and give the full paradigm for at least five clinical lexemes.
- State the orthographic rules governing English inflection — e-deletion, consonant doubling, y-to-i, the sibilant -es rule — precisely enough to be implemented, and give for each one a counter-example on which it over-generates.
- Identify the irregular forms in your corpus and explain why each must be listed in the lexicon rather than derived by rule.
- Measure the effect of inflection: report the type-to-lemma ratio of your corpus and the proportion of failures in your query set that inflection alone explains.

Evidence Expected

Precisely stated inflectional rules with paradigms, an irregular list, and a quantified measurement of the effect of inflection on retrieval.

5.1 Important terms

- Morpheme: Smallest unit of meaning, such as fracture or -s.
- Stem: Base form to which an affix is added, such as fracture.
- Affix: A morpheme attached to a stem, such as -s or -ed.
- Inflection: Changes a word's grammatical form without changing its basic word class.

- Paradigm: A set of grammatical forms of the same lexeme.

Inflection differs from derivation because inflection usually preserves the word class. For example, fracture and fractures are both nouns, while fracture and fractured may differ in grammatical function.

5.2 Common inflectional paradigms

Category	**Example forms**
Noun number	fracture, fractures
Verb tense	treat, treats, treated, treating
Verb aspect	operate, operating, operated
Agreement	patient reports, patients report
Adjective comparison	mild, milder, mildest

5.3 Clinical word paradigms

Base	**Inflected forms**
fracture	fractures, fractured, fracturing
treat	treats, treated, treating
operate	operates, operated, operating
diagnose	diagnoses, diagnosed, diagnosing
prescribe	prescribes, prescribed, prescribing

5.4 Orthographic rules

Rule	**Example**
Drop final e before -ing	operate → operating
Double final consonant	stop → stopped
Change y to i	injury → injuries
Add -es after sibilant sounds	diagnosis → diagnoses

Add -s in regular cases	fracture → fractures

These rules have exceptions. For example, play becomes playing, not plaing; irregular forms such as was and were should be stored separately.

5.5 Inflection and search

A search for fracture may fail to retrieve fractures, fractured, and fracturing if the system searches only exact forms.

A type-lemma ratio can be calculated as:

\[\text{Type-Lemma Ratio}= \frac{\text{Number of distinct word forms}} {\text{Number of distinct lemmas}} \]

A higher ratio indicates that more surface forms represent the same lemmas. The actual ratio and percentage of query failures must be calculated from the clinical corpus.

TASK 6 – Derivational Morphology 4 Marks

Guiding Enquiry

How does derivation build the specialised clinical vocabulary, and why is it harder to handle than inflection?

Investigate and interpret:

- Define derivation and list at least fifteen derivational affixes attested in your corpus — for example -itis, -ectomy, -ology, -osis, -aemia, -ation, -able, -ive, nephro-, hyper-, hypo-, re-, un- — stating the category change each one causes.
- Build the derivational family of at least six clinical roots, showing the order in which affixes attach and identifying at least two orders that are impossible.
- Explain semantic non-compositionality with clinical examples in which the meaning of the derived word cannot be predicted from its parts, and state what this implies for a search system that stems aggressively.
- Compare the productivity of your affixes empirically: count types and tokens for each in the corpus and classify each affix as productive or fossilised, with the evidence for the classification.
- Decide whether derivational relatives should be conflated for search, citing at least two cases where conflation helps the clinician and two where it produces a clinically misleading match.

Evidence Expected

A documented derivational analysis with an affix inventory, productivity measurements and a reasoned position on conflation.

6.1 Derivational affixes

Derivation creates a new word or changes its meaning or grammatical category.

Affix	Meaning/example	Typical effect
-itis	gastritis	inflammation
-ectomy	nephrectomy	surgical removal
-ology	cardiology	field of study
-osis	fibrosis	condition
-aemia	anaemia	blood-related condition
-ation	ventilation	noun formation
-able	treatable	adjective formation
-ive	responsive	adjective formation
nephro-	nephrology	kidney-related
hyper-	hypertension	excessive/high
hypo-	hypoglycemia	low/under
re-	re-treatment	again
un-	untreated	not
-al	renal	adjective formation
-ic	cardiac	adjective formation

Some affixes preserve the word class, while others change it. For example, treat is a verb and treatment is a noun.

6.2 Derivational families

Root	Related clinical words
nephro	nephrology, nephrologist, nephritis, nephrosis, nephrectomy, nephropathy
cardio	cardiology, cardiac, cardiologist, cardiopathy, cardiomyopathy, cardiogram
gastro	gastritis, gastrology, gastrectomy, gastroenterology, gastropathy, gastric

neuro	neurology, neurologist, neuritis, neuropathy, neurogenic, neurosurgery
derm	dermatology, dermatitis, dermatologist, dermal, dermatosis, dermopathy
osteo	osteology, osteitis, osteotomy, osteopathy, osteoarthritis, osteogenic

Affix order matters. For example, nephro + ectomy forms nephrectomy, while arbitrary combinations such as ectomynephro are not valid clinical words.

6.3 Semantic non-compositionality

The meaning of a derived word is not always the simple sum of its parts.

For example, nephrology means the medical study of the kidney, not merely "kidney + study" in a general sense. Clinical dictionaries are needed to store the precise meaning.

6.4 Productivity and conflation

An affix is productive when it can be used to form many new words. Productivity can be estimated from the number of distinct types and their frequency in the corpus.

Beneficial conflation:

- fracture and fractures
- treat and treated

Potentially misleading conflation:

- nephritis and nephrectomy — inflammation versus surgical removal
- hypertension and hypotension — high versus low blood pressure

Therefore, derivational forms should not automatically be treated as equivalent search terms.

TASK 7 – Finite-State Morphological Parsing 5 Marks

Guiding Enquiry

How can a transducer relate a surface form to its stem and features, and run in both directions on the same machine?

Investigate and interpret:

- Define a finite-state transducer as a two-tape automaton and explain the lexical and surface tapes with one fully worked clinical example.
- Build your lexicon as an FST organised into continuation classes, covering at least three parts of speech together with the inflectional and derivational affixes of Tasks 5 and 6.

- Hand-trace the parsing of at least five surface forms, showing the state sequence, the symbols consumed and emitted on each tape, and the final analysis — for example fractures → fracture +N +PL.
- Run the same transducer in the generation direction, producing surface forms from stem-plus-feature input, and show explicitly that parsing and generation use one machine and not two.
- Handle ambiguity: give at least two surface forms with more than one analysis, show the transducer returning all of them, and state how a downstream component would choose between them.

Evidence Expected
A working transducer specification with traced analyses in both directions and an explicit treatment of ambiguity.

7.1 Definition

A Finite-State Transducer (FST) maps one string to another. In morphology, it maps between:

- Lexical representation: stem and grammatical features
- Surface representation: actual word form

Example:

fracture+N+PL ↔ fractures

7.2 Two-tape representation

Lexical tape	Surface tape
fracture + noun + singular	fracture
fracture + noun + plural	fractures
treat + verb + past	treated
treat + verb + present participle	treating

A continuation-class lexicon groups words that follow the same morphological rules. For example, regular nouns may use a noun-plural continuation class, while irregular nouns use a separate class.

7.3 POS categories

The FST should support at least:

- Noun: fracture, fractures
- Verb: treat, treated, treating

- Adjective: renal, cardiac

7.4 Sample morphological analyses

Surface form	Lexical analysis
fractures	fracture + N + PL
treated	treat + V + PAST
treating	treat + V + PROG
diagnoses	diagnosis + N + PL
renal	renal + ADJ

The FST can also generate surface forms from lexical forms. For example, treat + V + PAST generates treated.

7.5 Ambiguity

A word may have more than one analysis.

- treated: past tense verb or past participle
- wound: a physical injury or the past tense of wind

The system can use surrounding words, POS tagging, or clinical context to choose the most likely analysis.

TASK 8 – Orthographic Rules and Two-Level Morphology 4

Marks Guiding Enquiry

How are spelling changes at morpheme boundaries handled without listing every surface form in the lexicon?

Investigate and interpret:

- Explain why a lexicon transducer alone cannot produce correct surface spellings, using at least three examples in which plain concatenation yields the wrong string.
- Write at least five orthographic rules in the standard rewrite notation (a → b / c __ d), stating the context of each rule precisely.
- Implement each rule as a transducer and compose the rule transducers with the lexicon FST; draw the cascade and state the order dependencies between the rules.
- Contrast this sequential cascade with the parallel two-level (Koskenniemi) formulation, and explain the practical advantage each has over the other.
- Test the complete system on at least twenty forms chosen to exercise every rule,

tabulate successes and failures, and account for every failure.

Evidence Expected

Correct rule transducers, a documented composition order, and a tested cascade with full error accounting.

8.1 Why concatenation is insufficient

Simply joining a stem and suffix can produce incorrect spellings.

Examples:

- operate + ing should become operating
- stop + ed should become stopped
- injury + s should become injuries

8.2 Rewrite rules

The notation a → b / c__d means "replace a with b in the context between c and d."

Rule	Example
e → 0 / __ + ing	operate + ing → operating
y → i / consonant __ + es	injury + es → injuries
0 → e / sibilant __ + s	diagnosis + s → diagnoses
0 → p / stop + ed	stop + ed → stopped
0 → p / stop + ing	stop + ing → stopping

These are simplified illustrative rules; a complete system needs more precise contexts and exceptions.

8.3 Transducer cascade

A cascade applies multiple transducers in sequence:

1. Lexical lookup
2. Affix recognition
3. Orthographic adjustment
4. Surface-form generation

The output of one transducer becomes the input of the next.

8.4 Cascade vs. two-level morphology

- Cascade approach: Uses several separate transducers in a sequence.
- Two-level approach: Relates lexical and surface forms through parallel constraints.

A cascade is easier to build in stages. A two-level system can handle interacting spelling changes more consistently, but may be more complex to design.

8.5 Testing

Test at least 20 forms, including regular forms, spelling changes, irregular forms, and invalid forms. Record:

Input	Expected output	Actual output	Status
operate + ing	operating	To be tested	Pending
stop + ed	stopped	To be tested	Pending
injury + es	injuries	To be tested	Pending

Actual outputs must be filled in after implementation.

TASK 9 – The Porter Stemmer 5 Marks

Guiding Enquiry

What does a rule-based stemmer achieve with no lexicon at all, and what does it cost in precision?

Investigate and interpret:

- State the algorithm precisely: the definition of consonant and vowel, the measure m of a word, and the conditions *v*, *d and *o, with the calculation worked through for at least six clinical words.
- Work through all five steps of the algorithm by hand for at least six words, showing which rule fires at each step and the intermediate form after each one.
- Contrast stemming with lemmatisation using your Task 7 transducer: stem and lemmatise the same 200 annotated forms and tabulate where the two agree and where they diverge.
- Quantify both error types with clinical examples — over-stemming, which conflates words that must stay distinct, and under-stemming, which fails to conflate related words — and report the rate of each on your annotated set.
- Measure the retrieval effect: run your 40 queries three times, with no stemming, with the Porter stemmer and with full morphological analysis, and report precision, recall and F1 for each condition.

Evidence Expected

Correct hand-traces of the algorithm, a quantified error analysis, and a retrieval experiment

comparing stemming against full analysis.

9.1 Definitions

The Porter stemmer is a rule-based algorithm that removes common English suffixes to produce a stem.

- Consonant: A letter that is not treated as a vowel.
- Vowel: a, e, i, o, u; y depends on its position.
- Measure \(m\): Number of vowel-consonant sequences in the stem.
- \(*v*\): The stem contains a vowel.
- \(*d*\): The stem ends in a double consonant.
- \(*o*\): The stem ends in consonant-vowel-consonant, excluding w, x, and final y.

9.2 Sample clinical words

Word	Possible Porter stem
fractures	fractur
treated	treat
treating	treat
diagnosis	diagnosi
medications	medic
infections	infect

Porter stemming is not the same as dictionary lemmatization. The output may not be a valid English word.

9.3 Main Porter steps

1. Step 1a: Handle plural endings, such as -s, -ies, and -sses.
2. Step 1b: Handle -ed and -ing when the stem contains a vowel.
3. Step 1c: Change terminal y to i in suitable cases.
4. Step 2: Replace longer suffixes, such as -ational, with shorter forms.
5. Step 3: Remove or replace suffixes such as -icate, -ative, and -alize.

6. Step 4: Remove selected suffixes when the measure is sufficiently large.
7. Step 5: Remove final e or reduce a double l under specified conditions.

The algorithm should be run on the selected 200 annotated clinical forms to compare its results with the expected lemmas.

9.4 Over-stemming and under-stemming

- Over-stemming: Different words are reduced to the same stem, even though their meanings differ.
- Under-stemming: Related forms remain different and are not matched.

Examples of possible over-stemming include clinically different terms that happen to share a stem after suffix removal. Under-stemming may occur when irregular forms are not reduced to the same base.

9.5 Retrieval evaluation

For the 40 clinician queries, compare:

1. No stemming
2. Porter stemming
3. FST-based morphological normalization

Calculate:

\[Precision=\frac{TP}{TP+FP} \]

\[Recall=\frac{TP}{TP+FN} \]

\[F1=\frac{2 \times Precision \times Recall}{Precision+Recall} \]

Use clinician relevance judgments to determine TP, FP, and FN. Actual scores cannot be stated without the evaluation results.

TASK 10 – Implement and Test the Pipeline 3 Marks

Guiding Enquiry

Can a working implementation reproduce the behaviour you derived by hand? **Investigate and interpret:**

- Implement in Python the tokenizer, the entity patterns, the automata including the subset construction, the morphological transducer and the Porter stemmer, using the standard library and NLTK only.

- Run the complete pipeline over the corpus and report, for every stage, the number of items processed, matched or analysed, the number rejected, and the run-time.
- Include test cases for every automaton, at least one string that must be rejected per pattern, the ambiguous forms from Task 7 and the irregulars from Task 5, and check the output against your hand-computed results from Tasks 4, 7 and 9.
- Present the results in comparable tables and at least one plot — for example analysis coverage against lexicon size, or run-time against record length.

Evidence Expected

Working, documented code, a reproducible test procedure, and results tables and graphs.

10.1 Pipeline

The proposed system contains the following stages:

Clinical Records

↓

Regex Tokenisation

↓

Text Normalisation

↓

Finite-State Validation

↓

Morphological Analysis (FST)

↓

Stemming / Lemma Mapping

↓

Clinical Search Index

↓

Search Results

10.2 Python implementation outline

Python's standard library provides re for regular expressions. NLTK can provide a Porter stemmer.

import refrom nltk.stem import PorterStemmerstemmer = PorterStemmer()text = "Patient

fractures were treated."tokens = re.findall(r"[A-Za-z]+|\d+(?:\.\d+)?", text.lower())stems = [stemmer.stem(token) for token in tokens]print(tokens)print(stems)

This is a basic demonstration. The complete assignment implementation should also include entity regexes, automata, morphological rules, irregular forms, and search indexing.

10.3 Testing plan

Test:

- Every automaton with valid and invalid inputs.
- Regular and irregular morphological forms.
- Ambiguous words.
- Unseen or out-of-vocabulary words.
- Tokenisation of hyphens, abbreviations, decimals, and attached units.
- Search queries with and without normalization.

10.4 Evaluation table

Stage	Records/tokens processed	Matched	Rejected	Runtime
Tokenisation	To measure	To measure	To measure	To measure
Regex validation	To measure	To measure	To measure	To measure
Morphological analysis	To measure	To measure	To measure	To measure
Search evaluation	40 queries	To measure	To measure	To measure

The counts and runtime must be recorded during actual execution.

TASK 11 – Compare, Recommend and Defend the Normalisation Design 2

Marks Guiding Enquiry

Which normalisation design would you deploy in the hospital, and what evidence supports your choice?

Investigate and interpret:

- Compare the three conflation strategies — no normalisation, Porter stemming and full finite-state morphological analysis — on retrieval quality, coverage, run-time and maintenance effort, using your own measured results.
- Recommend a design, stating its expected accuracy, its failure modes, and what a

clinician would actually notice when it fails.

- Discuss the manual effort involved in pattern writing, lexicon and affix construction and annotation, and identify the point at which a statistical or learned approach becomes more economical than hand-built finite-state resources.

Evidence Expected

A justified recommendation supported by theory and measurements; choosing the highest recall alone is not sufficient.

11.1 Comparison

Method	Quality	Coverage	Runtime	Maintenance
No normalization	Low for variant forms	Exact forms only	Low	Easy
Porter stemmer	Moderate	Common English suffixes	Low	Easy
Full morphological FST	Potentially high for covered clinical forms	Depends on lexicon and rules	Moderate	Requires expert updates

11.2 Recommendation

A morphology-aware FST combined with regex validation is recommended for the hospital search system.

It can map inflectional variants such as fracture, fractures, and fractured to a common lemma while preserving important clinical distinctions. Regex validation can help recognize structured values such as dates, patient IDs, and dosage formats.

However, the system must not merge clinically different terms such as hypertension and hypotension. Regular review by clinical experts is needed to maintain the lexicon and rules.

The final recommendation should be supported by measured precision, recall, F1 score, coverage, runtime, and maintenance effort.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Changed Requirement

The system must now ingest two new sources. The first is optical character recognition output from scanned handwritten forms, in which characters are routinely confused (0 for O, 1 for l,

rn for m), words are broken across line endings, and diacritics are lost. The second is notes typed by staff in Romanised regional language mixed with English, for example “patient ku fever irundhuchu, paracetamol 500mg kuduthen”. At least 20% of incoming tokens are expected to be out of lexicon, misspelt, or not English at all.

Re-evaluate your original design under the changed conditions. You

may: **RETAIN → MODIFY → or REDESIGN**

your original solution.

A. Impact Analysis – 2 Marks

Guiding Enquiry

What exactly has changed, and which of your original assumptions no longer hold?

- Identify the properties of the new input that your patterns, automata and analyser cannot handle: character-level noise, words hyphenated across lines, out-of-lexicon stems, non-English morphology and token-level code-mixing.
- Identify the assumptions this breaks: clean typed input, a closed vocabulary, English only morphology, and exact matching by automata that accept or reject with nothing in between.
- Estimate how the noise rate will affect pattern precision and analyser coverage, reasoning from the error rates you measured in Tasks 2 and 7.

Evidence Expected

A clear analysis of how the new input exposes gaps in the original design.

B. Evaluation of the Original Design – 2 Marks

Guiding Enquiry

How would your recommended pipeline actually behave on the new input?

- Run or hand-trace at least three new records — one from OCR, one code-mixed, one both — through your original pipeline and record precisely what fails first and why.
- Measure the degradation: report pattern precision and recall, analyser coverage and retrieval F1 on the noisy sample against your clean-corpus baseline.
- Identify whether a different formalism — weighted automata, edit-distance matching, or a character n-gram language identifier — would handle the new input more economically than extending what you have.

Evidence Expected

A technically supported evaluation of the original design with traces or measurements.

C. Adaptation or Redesign – 2 Marks

Guiding Enquiry

Should the finite-state layer be extended, replaced or supplemented?

- Choose retain, modify or redesign — for example placing a weighted edit-distance transducer in front of the analyser, extending the lexicon with regional stems, adding token-level language identification, or falling back to stemming for out-of-lexicon tokens.
- Describe the revised design and state exactly what changes in the patterns, the

automata, the lexicon and the rule cascade.

- Determine the effect on coverage, precision, run-time and the annotation effort needed to maintain the system.

Evidence Expected

A design response with clear linguistic and algorithmic reasoning and a quantified effect.

D. Evidence-Based Defence – 2 Marks

Guiding Enquiry

Why is your final decision the right one, given the evidence you generated?

- Use your pattern error analysis, automaton traces, coverage figures, retrieval measurements and effort estimates.
- Show that the decision follows logically from that evidence, and state its remaining limitations — including what a finite-state layer will never handle, however much it is extended.

Evidence Expected

A well-supported final decision integrating theory, experiments and the changed requirements.

A. Impact of OCR errors and code-mixing (2 Marks)

OCR and code-mixed text can reduce the accuracy of the original normalization system.

Common OCR errors:

OCR error	**Example**
0 confused with O	1O mg instead of 10 mg
1 confused with l	l0 mg instead of 10 mg
rn confused with m	rnild instead of mild
Broken words	nephro + logy
Missing diacritics	A regional-language word loses accent marks

Code-mixed notes may contain English clinical terms combined with Romanized regional-language words. A standard English lexicon may fail to recognize those forms.

The 20% of out-of-vocabulary, misspelled, or non-English tokens can lead to:

- Incorrect token boundaries

- Failed regex matches
- Incorrect morphological analyses
- Missed search results
- False matches caused by overly broad normalization

B. Evaluate the original system on three new records

Select three additional records containing OCR errors, misspellings, or code-mixed text.

Record	**Input issue**	**Expected result**	**Original system result**
Record 1	1O mg OCR error	Recognize 10 mg if context supports it	To test
Record 2	Broken word such as nephro logy	Recognize intended term if safely recoverable	To test
Record 3	Romanized regional-language phrase	Identify supported clinical terms	To test

Calculate the same metrics used earlier:

$$ Precision=\frac{TP}{TP+FP} $$

$$ Recall=\frac{TP}{TP+FN} $$

$$ F1=\frac{2PR}{P+R} $$

The actual scores must be calculated after testing the three records. The system should not automatically correct uncertain medical terms without retaining the original text or providing a confidence score.

C. Modify or redesign the system

The original pipeline can be modified as follows:

OCR / Clinical Text

↓

OCR Error Detection

↓

Character-Level Correction

↓

Language Identification

↓

Regex Tokenisation and Validation

↓

Morphological FST

↓

Clinical Search Index

Recommended changes:

1. OCR correction: Use context-aware alternatives for commonly confused characters such as 0/O, 1/l, and rn/m.
2. Edit distance: Compare unknown words with known clinical terms and suggest close matches.
3. Character n-grams: Help identify misspelled or partially broken words.
4. Language identification: Detect English and supported regional-language portions.
5. Expanded lexicon: Add frequently used Romanized regional-language clinical expressions.
6. Confidence threshold: Automatically correct only high-confidence cases; send uncertain cases for review.
7. Original-text retention: Keep the source text so that corrections can be checked.

D. Defend the redesigned system

The modified system is expected to improve recall for OCR errors and spelling variants. However, aggressive correction may increase false positives.

For example, an OCR correction from 1O mg to 10 mg may be reasonable in context, but an incorrect correction to a medication strength could be dangerous. Therefore, medical values and drug names require stricter validation.

The redesigned system should be evaluated using the same three records and the same relevance

judgments as the original system. Compare precision, recall, F1 score, coverage, and runtime.

Measure	**Original system**	**Modified system**
Precision	Measure	Measure
Recall	Measure	Measure
F1 score	Measure	Measure
OOV coverage	Measure	Measure
Runtime	Measure	Measure

MARK DISTRIBUTION

Task	Marks
Task 1 – Frame the Problem: Models, Algorithms and Levels of Analysis	3
Task 2 – Basic Regular Expression Patterns	4
Task 3 – Regular Expressions for Tokenisation and Normalisation	4
Task 4 – Finite State Automata: Design, Non-determinism and Equivalence	5
Task 5 – Inflectional Morphology	3
Task 6 – Derivational Morphology	4
Task 7 – Finite-State Morphological Parsing	5
Task 8 – Orthographic Rules and Two-Level Morphology	4
Task 9 – The Porter Stemmer	5
Task 10 – Implement and Test the Pipeline	3
Task 11 – Compare, Recommend and Defend the Normalisation Design	2
Subtotal	**42**
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**